

保存文件类型、文件大小、拥有者和拥有组、访问权限、链接数、访问和修改时间等基本元数据，以及文件内容所在位置的索引。对 inode 的一致性检查需要保证这些信息合理且与文件系统中保存的其他信息一致。举例来说，inode 中记录的占用块的数量应与其实际索引的块数量相一致；inode 中保存的链接数应与整个文件系统中保存的指向该 inode 的目录项的个数一致；对于索引中用到的所有索引块和数据块，都应有合理的块号（如块号不应超过文件系统中的最大编号），在分配表中应为被分配的状态，且同一个块不应该被其他 inode 使用。

与超级块一样，inode 中的部分信息是无法通过文件系统中保存的内容进行检查和修复的。这部分信息包括 inode 中保存的拥有者，以及创建、访问和修改时间等。对于这部分元数据，e2fsck 在进行系统一致性检查时，只能进行基本的合理性检查，保证这部分元数据的数值处在合理的范围内。

目录

目录是一种特殊的文件。除了对其 inode 进行检查之外，还应检查其中保存的目录项的一致性。目录项中保存文件名和文件名所对应的 inode 号。在进行检查时，需保证文件名和目录项的长度均在有效且合理的范围内，并且目录项的长度应不小于文件名的长度与 inode 号所占用的空间之和。对于 inode 号，检查时需要保证每个 inode 号在合理且有效的范围内，并且其对应的 inode 在 inode 分配表中标记为已使用。此外，每个目录应至少包含“.”和“..”两个特殊的目录项，且对应的 inode 号应为当前目录和父目录的 inode 号。

目录项的检查与此前介绍的 inode 的检查需要协同进行，以统计每个 inode 的实际链接数，即指向 inode 的目录项个数。在目录项的统计工作完成后，需要按照实际链接数对不同 inode 进行处理。对于链接数为零的 inode，由于从文件系统的根无法访问到该 inode，因此可以在检查后将其删除。对于链接数不为零，却无法从文件系统根出发被访问到的“孤立”inode，在检查后，e2fsck 会将其保存在特殊的目录“lost+found”中。

12.2.3 fsck 的局限和问题

fsck 是在一致性问题发生之后进行补救的工具，虽然具有一定的一致性检查和修复能力，但并不能处理所有的一致性问题。在一些情况下，fsck 需要以交互的方式在用户的指导下完成修复过程；而在另外一些情况下，它对用户丢失的数据也束手无策。

此外，fsck 还存在效率问题。由于内存的容量一般远远小于存储设备的容量，将存储设备中的所有文件数据读取到内存中进行检查是不现实的。因此 fsck 在进行检查和修复时，一般需要多次扫描存储设备以把需要的部分数据加载到内存中进行访问。由于存储设备的访问比较耗时，一次 fsck 经常需要几小时甚至几十小时。另外，在 fsck 进行修复时，如果计算机系统再次发生故障，情况会变得更加复杂。

相对于 fsck 这种在发生损坏后进行检查和修复的方法，预防性地进行文件系统一致

性保护往往能够获得更好的效果。在接下来的几节中，我们将介绍几种用于文件系统的常见系统性方法，包括日志、写时拷贝和 Soft updates。

12.3 原子更新技术：日志

本节主要知识点

- 日志如何保证多个修改同时持久化？
- 日志的方法可以分为哪几类？
- 一个日志的生命周期是怎样的？
- 日志是如何在真实文件系统中应用的？

当有多件事情需要处理时，为了防止遗漏，我们通常会把所有事情先记录在一个待办清单中。这样即使被其他突发情况打断，我们也可以通过检查待办清单，知道有哪些事情是还未进行的，并将其完成。同理，当文件系统想要原子地完成多个修改时，为了防止只完成了部分修改就发生崩溃等情况，其先将这些待做的修改（即“日志”）记录在一个专有的存储区域中。一旦发生故障，文件系统可以通过读取日志将未完成的修改继续完成。

12.3.1 日志机制的原理

在文件系统中使用的日志一般为预写式日志（Write-Ahead Logging, WAL）^[3]。其原本用于在数据库系统中提供数据的原子持久性。正如其名，预写式日志要求在进行修改之前先在日志中记录修改的内容，以便在发生崩溃等情况时通过日志进行数据恢复，保证多个修改的原子性。

日志的生命周期

每一条日志信息从创建到销毁会经过一系列步骤和状态，一般包括：

- 创建：在使用日志之前，需要先进行日志的创建和初始化操作。在此过程中，文件系统会为日志分配内存和存储空间，并初始化维护日志所需的元数据。此后，日志进入写入阶段。
- 写入：在写入阶段，文件系统可以将要进行的操作或操作影响的数据及其所在位置写入日志中。举例来说，如果文件系统想要将位置 0x50 处的数据 A 改成 B，需要在日志中记录“位置 0x50 上的数据从 A 修改为 B”。需要注意的是，由于操作数量较多或者操作的数据量较大，在写入阶段记录的操作信息（即日志内容）往往超出存储设备的原子写入大小，因而无法原子地写入存储设备中。因此，在崩溃后，存储设备中处于写入阶段的日志内容可能是不完整的。为了防止这些不

完整的日志内容造成一致性问题，在进行恢复时，处于写入阶段的日志的内容会被直接丢弃。换句话说，在写入阶段的日志内容不会马上生效。这些日志内容的生效需要等到日志的提交阶段。

- **提交**：在提交阶段，文件系统需要将此前在日志中记录的操作信息原子地标记为有效。在日志提交成功后，日志中所记录的信息在进行崩溃恢复时才会发挥效用。具体来说，在进行提交前，文件系统需要将日志内容以固定的格式持久且完整地保存在存储设备中。在保证所有日志内容均已持久化在存储设备中之后，日志系统将在存储设备上原子地标记日志为已提交状态，并进入完成阶段。
- **完成**：在完成阶段，文件系统可以将实际的修改写回存储设备中。由于这些写回的位置及其信息均在日志中有所记录，若在此阶段发生崩溃，在进行恢复时，会使用日志中记录的内容进行恢复，保证日志中所记录操作的原子性。
- **无效**：当文件系统已经完成所有需要进行的修改，并且保证这些修改已经持久化之后，其将日志标记为无效。标记为无效的日志在进行恢复时不会进行恢复操作。
- **销毁**：日志的记录需要占用一定的内存和存储资源。日志无效后，文件系统可以对日志所占用的这些资源进行回收，即销毁日志。一般情况下，这一过程可以通过批量化的形式延迟完成。

日志的内容

根据日志中具体保存的内容和恢复时进行的操作，日志可以分为重做日志（Redo Logging）和撤销日志（Undo Logging）两种。

- **重做日志**。重做日志将数据位置和此位置上即将被写入的新数据（即修改后的数据）保存在日志中。当需要原子完成的多个修改所对应的位置和新数据被保存到日志中之后，重做日志可以进行提交。一旦日志成功提交，可以认为此日志中的所有操作已经持久化完成。在提交之后，用户可以开始进行真正的修改操作，将修改后的内容保存在原位置上。在恢复时，会将已经提交的日志中所记录的新的数据重新应用到其原本应该在的位置上。

在使用重做日志时，日志的成功提交标志着日志中修改的成功持久化。只要日志已经提交，不管其修改是否已经写回原位置，这些修改操作均可以从日志中恢复。因此对于重做日志来说，文件系统可以不急于将修改写回原位置。重做日志可以长期停留在完成（即已提交）状态，日志的无效化和销毁过程可以延后。

- **撤销日志**。与重做日志不同，撤销日志会在日志中记录数据位置和位置上原本保存的数据。撤销日志相当于对数据提前进行一次备份，一旦后续操作发生问题，便恢复到备份时的状态。在进行一组需要原子完成的操作之前，用户需要先将所有要修改的位置及其位置上原本存放的数据提前保存在日志中，并进行提交。当日志提交后，用户可以进行这一组修改操作，无须关心操作的原子性。一旦日志提交后发生崩溃，日志系统会从日志中读取崩溃前可能修改的位置，并将这些位

置上的数据恢复到日志中记录的原本状态。

与重做日志不同，在使用撤销日志时，数据修改的持久化取决于日志是否被标记为无效。在撤销日志提交后，只要其还未被标记为无效，在日志所记录的位置上的任何数据修改，均会在崩溃恢复时回滚到日志刚提交时的状态。因此，当一个操作的所有修改被应用到其原本位置，并都持久保存下来之后，用户需要将日志标记为无效，从而保证这些修改不会在发生崩溃后被日志恢复过程撤销。

12.3.2 日志的批量化与合并优化

一般来说，由于存储设备的访问速度较慢，如果每个文件系统操作之后都进行日志的提交，每个操作均会产生对存储设备的访问，影响文件系统的性能。正如同在文件系统中可以使用页缓存异步地将数据从内存写回存储设备，日志也可以异步延迟提交。

日志的延迟提交策略可以将多份日志进行批量化提交，从而减少对存储设备的访问次数。同时，如果多份日志中包含相关的内容，日志内容还可以进行合并，从而减少提交日志的大小，进而减少写入存储设备的数据量。例如，用户先创建文件 A 并写入了大量数据，随后又删除了文件 A。如果每次操作都使用日志进行提交，那么会有至少三份日志：文件创建、文件数据修改和文件删除。而如果将这几个操作合并在一起，其实日志中只需修改相应的时间记录（如父目录的修改时间）即可。具体的文件创建、删除操作全都可以相互抵消，从而大幅减少实际写入存储设备的数据量。又如，用户对文件 A 的前 1KB 数据进行修改后，又对文件 A 相同位置的数据进行了第二次修改。由于第二次的修改内容覆盖了第一次的内容，因此日志合并后只需保留第二次修改的数据即可。

12.3.3 日志应用实例：JBD2

JBD2 (Journaling Block Device 2)^[4] 是 Linux 系统中一个通用的日志组件。JBD2 通过统一的接口为 Linux 内核中的其他模块提供日志功能。例如，Linux 中的 Ext4 文件系统使用 JBD2 保存和管理日志。

在介绍 JBD2 之前，先介绍 JBD2 中的几个概念。

- 日志 (Journal)：表示整个记录在系统中的日志。
- 原子操作 (Handle)：表示一系列需要原子完成并持久化的修改，一般使用 handle 作为标识。
- 事务 (Transaction)：由一个或多个原子操作组成，这些原子操作将被共同提交。

在前文中我们曾经提到，日志可以进行异步提交，且在提交之前所有的日志记录甚至不需要写入存储设备。如图 12.2 所示，在 JBD2 的设计中，通过利用内存中的页缓存等已有机制在内存中暂时保存数据（即图中的“内存”部分），并通过保存内存指针的方式将一个原子操作涉及的多个内存缓存结构组织在一起。

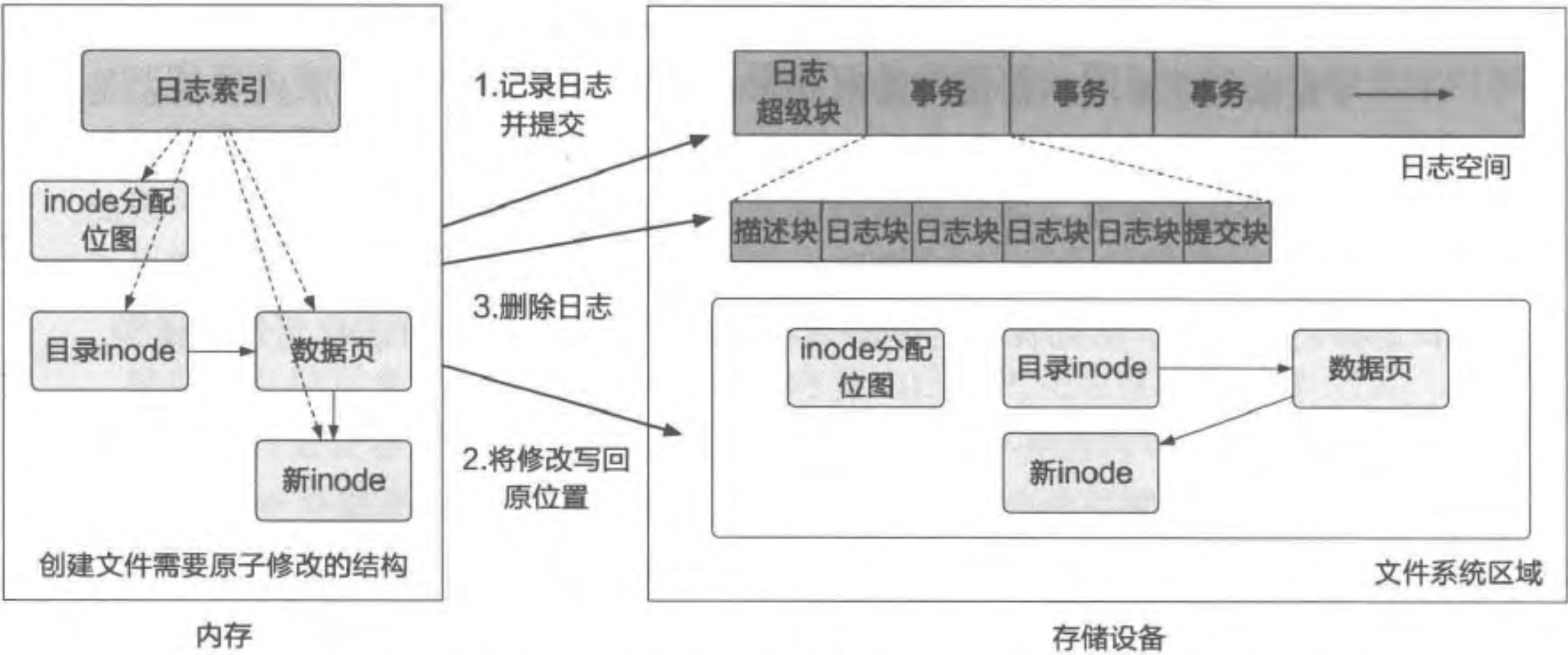


图 12.2 JBD2 中的日志，其中深色区域为 JBD2 在存储设备上的存储格式

JBD2 的存储格式

图 12.2 中给出 JBD2 的存储格式。JBD2 的日志可以保存在现有文件的文件中或者直接保存在设备上。每个 JBD2 的日志都有一个超级块。类似文件系统中的超级块，JBD2 中的超级块保存了整个 JBD2 日志的元数据，包括整个日志空间的大小和日志的起始位置等。超级块之后的日志空间中保存一个个事务。事务的开头保存事务的描述块，描述块中保存整个事务的信息，其内容为一系列标签（tag）。每个标签按照顺序对应描述块之后的一个日志块。标签中记录了日志块对应的数据的位置（块号）和校验码（用于保证内容完整性）等元数据。描述块之后的日志块则保存具体的数据，其数量与描述块中的标签数量相对应。每个日志的最后是一个提交块，其中记录着整个事务的提交状态。

JBD2 的基本操作

要在文件系统中使用 JBD2 管理日志，需要首先初始化日志系统。代码片段 12.1 中的 `jbd2_journal_init_inode` 函数初始化一个保存在文件中的 JBD2 日志系统，其参数 `inode` 表示日志保存在该 `inode` 对应的文件中。一个 `journal_t` 类型的内存结构作为该函数的返回值，代表此 JBD2 日志系统。在该 JBD2 日志系统上的操作均需要使用该结构完成。在初始化日志系统之后，一般还需要调用 `jbd2_journal_load` 函数尝试读取和恢复文件中已有的日志。这些读取和恢复操作一般在文件系统挂载时进行。相对地，`jbd2_journal_destroy` 函数销毁日志系统在内存中占用的结构，一般在文件系统卸载时进行。

代码片段 12.1 JBD2 的部分操作接口

```
1 // 初始化日志系统 (日志存在 inode 表示的文件中)
2 journal_t *jbd2_journal_init_inode(struct inode *inode);
```

```

3
4 // 读取并恢复已有日志 (如果存在)
5 int jbd2_journal_load(journal_t *journal);
6
7 // 销毁内存中日志系统的信息
8 int jbd2_journal_destroy(journal_t *journal);

```

代码片段 12.2 给出了 JBD2 所提供的事务和原子操作的接口。文件系统可以使用 JBD2 的原子操作保证文件操作中多个步骤的原子性。代码片段 12.3 给出了一个使用 JBD2 进行文件创建的例子。首先，文件系统通过 `jb2_journal_start` 创建一个新的原子操作（对应日志生命周期中的创建阶段，此后日志进入写入阶段）。新创建的原子操作会被加入当前事务之中。若当前不存在事务，则首先创建一个新的事务作为当前事务，再将原子操作加入其中。此后，文件系统开始创建文件。首先将分配得到的 `inode` 标记为占用（第 5 ~ 9 行）。具体过程是，首先获得对应的 `inode` 分配信息所在的块的缓存（`bitmap_bh`），然后通过 `jb2_journal_get_write_access` 函数通知 JBD2 即将对该块进行修改（第 7 行），将块中对应的位置 1（第 8 行），最后通过 `jb2_journal_dirty_metadata` 表示此块中的数据已经进行了修改（第 9 行）。使用类似的过程，在第 11 ~ 15 行对 `inode` 结构进行初始化，在第 17 ~ 21 行将目录项写入对应目录中。当创建文件的这些步骤完成后，文件系统通过调用 `jb2_journal_stop` 来通知 JBD2 该原子操作结束（表示日志的写入阶段结束，可以进入日志的提交阶段）。事务的管理和提交一般由 JBD2 自动完成，一个事务会在等待足够长的时间或足够多的原子操作之后才会进行提交。但文件系统也可以通过调用 `jb2_journal_force_commit` 来强制提交当前事务。

代码片段 12.2 JBD2 的部分事务和原子操作接口

```

1 // 在当前事务中开始一个新的原子操作
2 handle_t *jb2_journal_start(journal_t *journal, int nblocks);
3
4 // 通知 jbd2 即将修改缓冲区 bh 中的元数据
5 int jbd2_journal_get_write_access(handle_t *handle, struct buffer_head *bh);
6
7 // 通知 jbd2 即将使用一个新的缓冲区
8 int jbd2_journal_get_create_access(handle_t *handle, struct buffer_head *bh);
9
10 // 通知 jbd2 该缓冲区中包含脏元数据
11 int jbd2_journal_dirty_metadata(handle_t *handle, struct buffer_head *bh);
12
13 // 结束一个原子操作
14 int jbd2_journal_stop(handle_t *handle);
15
16 // 强制提交当前事务
17 int jbd2_journal_force_commit(journal_t *journal);

```

代码片段 12.3 使用 JBD2 进行文件创建的例子

```
1 handle_t *handle;
2 // 原子操作：创建新文件
3 handle = jbd2_journal_start(journal, nblocks=8);
4
5 // 1. 标记 inode 为占用
6 bitmap_bh = read_inode_bitmap(sb, group);
7 jbd2_journal_get_write_access(handle, bitmap_bh);
8 set_bit(ino, bitmap_bh->b_data);
9 jbd2_journal_dirty_metadata(handle, bitmap_bh);
10
11 // 2. 初始化 inode
12 inode_bh = get_inode_bh(sb, ino);
13 jbd2_journal_get_write_access(handle, inode_bh);
14 init_inode(inode_bh);
15 jbd2_journal_dirty_metadata(handle, inode_bh);
16
17 // 3. 将目录项写入目录中
18 data_bh = get_data_page(dir_inode);
19 jbd2_journal_get_write_access(handle, data_bh);
20 add_dentry_to_data(page, filename, ino);
21 jbd2_journal_dirty_metadata(handle, data_bh);
22
23 jbd2_journal_stop(handle); // 结束原子操作
```

Ext4 的日志选项

使用日志会导致文件系统中的修改被写入存储设备两次——第一次写入日志区域，第二次写入数据原本应该写入的位置。当需要修改的数据量较大时，存储设备的两次写入开销会影响文件系统性能。为此，文件系统一般会在日志保证的崩溃一致性和性能之间进行权衡。例如，Ext4 文件系统对于元数据修改默认使用日志进行一致性保护，但对于文件数据部分，其提供 writeback、ordered 和 journal^[5-6] 三种选项（如表 12.2 所示）。

表 12.2 Ext4 文件系统中的日志选项

日志选项	数据是否使用日志保证	顺序要求	性能	一致性
writeback	否	无要求	最好	弱
ordered	否	数据在元数据之前写回	中等	中等
journal	是	数据和元数据均在日志中	较差	强

- 写回模式（Writeback Mode）。在该模式下，Ext4 对于文件数据部分的修改不进行日志记录，且对文件数据的写回没有特定的要求。在这种模式下，文件的数据块修改可以在任何时候写入磁盘中进行持久化。这种模式的一致性保证比较差。在发生崩溃的时候，有更大的概率发生不一致的情况。但是由于对写回顺序没有特殊要求，这种模式的性能往往是最优的。
- 顺序模式（Ordered Mode）。在该模式下，Ext4 对于文件数据部分的修改同样不进

行日志记录，但是对文件数据何时能够写回磁盘中进行持久化有特定的顺序要求。对于文件的数据和元数据修改，需要保证文件数据部分的修改在文件元数据被持久化前持久化到设备中。这一顺序的保证可以减少数据不一致的情况出现，增强文件系统的一致性保证。但对于跨多个存储块的数据写入，顺序模式无法保证这些数据持久化的原子性。因而如果在对文件中一段数据进行覆盖写入时发生崩溃，这一段数据可能一半是新的，另一半是旧的。该模式是 Ext4 中的默认模式，也是一种性能和一致性保证之间的权衡。

- 日志模式 (Journal Mode)。在该模式下，Ext4 文件中的数据和元数据修改均使用日志进行保护。这是一种一致性保证很强的模式，但是要求文件数据在日志中和原位置上进行两次写入。因此，对于产生大量文件数据修改的场景来说，这种模式会带来不小的性能开销和不必要的写入操作。

12.3.4 讨论和小结

小思考

使用日志有哪些优点和缺点？什么情况下适合使用日志进行一致性保证？

日志是一种保证原子更新和崩溃一致性的常用方法。通过日志记录，可以保证任意位置、任意数量操作的原子完成，因而应用范围比较广泛，特别适用于修改位置比较分散的场景。然而，使用日志同样会带来一些问题。首先，日志要求所有的修改先在日志中进行记录，再将修改应用到其原有位置上。这样所有的修改都执行了两次：一次在日志中，另外一次在原位置上。因此，对于数据修改较多的场景，使用日志保证原子性可能会带来很大的写入开销。其次，日志的原子性保证依赖于日志恢复，因此在发生崩溃并重启后，需要首先进行日志恢复。只有日志恢复完毕之后，文件系统才能开始处理新的请求。若日志恢复时间较长，整个文件系统将长时间处于不可用状态，进而影响操作系统或应用程序的启动时间。最后，在使用日志时，需要为日志预留一定的空间。在使用日志时，这部分空间会被反复写入和修改。对于闪存等容易被写穿的存储介质来说，日志区域将导致比较严重的磨损问题。

12.4 原子更新技术：写时拷贝

本节主要知识点

- 写时拷贝技术如何保证多处数据的原子更新？
- 文件系统如何使用写时拷贝技术保证崩溃一致性？